

Part 1 - Build Your Own Practice Schema

Before attempting the tasks below, design and create your own small database. Do not copy a schema from anywhere - write your own CREATE TABLE statements based on the description below, then insert your own sample data using INSERT statements.

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

Customers:

Query #1 Execution time: 12.05ms

id	name	email	signup_date
1	Dhruv	dhruv@gmail.com	2025-01-01
2	Aayush	aayush@gmail.com	2025-01-02
3	Kirish	kirish@gmail.com	2025-01-03
4	Jaymin	jaymin@gmail.com	2025-01-04
5	Raj	raj@gmail.com	2025-01-05
6	Muel	muel@gmail.com	2025-01-06
7	Harsh	harsh@gmail.com	2025-01-07
8	Yaash	yaash@gmail.com	2025-01-08
9	Dev	dev@gmail.com	2025-01-09
10	Parth	parth@gmail.com	2025-01-10
11	Nisarg	nisarg@gmail.com	2025-01-11
12	Manav	manav@gmail.com	2025-01-12
13	Rohan	rohan@gmail.com	2025-01-13
14	Aryen	aryen@gmail.com	2025-01-14
15	Karan	karan@gmail.com	2025-01-15
16	Om	om@gmail.com	2025-01-16
17	Vivek	vivek@gmail.com	2025-01-17
18	Rahul	rahul@gmail.com	2025-01-18
19	Sahil	sahil@gmail.com	2025-01-19
20	Ankit	ankit@gmail.com	2025-01-20

Show details

Accounts:

Query #1 Execution time: 0.12ms

id	name	balance
1	Alice	5000.0
2	Bob	3000.0

Restorents:

Query #3 Execution time: 0.13ms

id	name	cuisine
1	Dominos	Pizza
2	Pizza Hut	Pizza
3	McDonalds	Fast Food
4	Burger King	Fast Food
5	Subway	Sandwich
6	La Pinoz	Italian
7	KFC	Chicken
8	Havmor	Ice Cream
9	Honest	Gujarati
10	Sankalp	South Indian

Orders:

Query #4 Execution time: 0.22ms

id	customer_id	restaurant_id	total	status	created_at
1	1	1	350.0	Delivered	2025-06-01 10:00:00
2	2	2	450.0	Pending	2025-06-01 11:00:00
3	3	3	220.0	Delivered	2025-06-01 12:00:00
4	4	4	310.0	Cancelled	2025-06-01 13:00:00
5	5	5	180.0	Delivered	2025-06-01 14:00:00
6	6	6	540.0	Pending	2025-06-01 15:00:00
7	7	7	600.0	Delivered	2025-06-01 16:00:00
8	8	8	150.0	Delivered	2025-06-01 17:00:00
9	9	9	270.0	Cancelled	2025-06-01 18:00:00
10	10	10	320.0	Delivered	2025-06-01 19:00:00
11	11	1	430.0	Pending	2025-06-02 10:00:00
12	12	2	510.0	Delivered	2025-06-02 11:00:00
13	13	3	290.0	Delivered	2025-06-02 12:00:00
14	14	4	410.0	Cancelled	2025-06-02 13:00:00
15	15	5	240.0	Delivered	2025-06-02 14:00:00
16	16	6	630.0	Pending	2025-06-02 15:00:00
17	17	7	720.0	Delivered	2025-06-02 16:00:00
18	18	8	160.0	Delivered	2025-06-02 17:00:00
19	19	9	330.0	Pending	2025-06-02 18:00:00
20	20	10	450.0	Delivered	2025-06-02 19:00:00
21	1	2	390.0	Delivered	2025-06-03 10:00:00
22	2	3	260.0	Pending	2025-06-03 11:00:00
23	3	4	340.0	Delivered	2025-06-03 12:00:00
24	4	5	190.0	Cancelled	2025-06-03 13:00:00
25	5	6	560.0	Delivered	2025-06-03 14:00:00
26	6	7	690.0	Delivered	2025-06-03 15:00:00
27	7	8	170.0	Pending	2025-06-03 16:00:00
28	8	9	290.0	Delivered	2025-06-03 17:00:00
29	9	10	410.0	Cancelled	2025-06-03 18:00:00
29	9	10	410.0	Cancelled	2025-06-03 18:00:00
30	10	1	350.0	Delivered	2025-06-03 19:00:00
31	11	2	480.0	Pending	2025-06-04 10:00:00
32	12	3	230.0	Delivered	2025-06-04 11:00:00
33	13	4	370.0	Delivered	2025-06-04 12:00:00
34	14	5	200.0	Cancelled	2025-06-04 13:00:00
35	15	6	590.0	Delivered	2025-06-04 14:00:00
36	16	7	710.0	Pending	2025-06-04 15:00:00
37	17	8	180.0	Delivered	2025-06-04 16:00:00
38	18	9	300.0	Delivered	2025-06-04 17:00:00
39	19	10	440.0	Pending	2025-06-04 18:00:00
40	20	1	360.0	Delivered	2025-06-04 19:00:00
41	1	3	250.0	Delivered	2025-06-05 10:00:00
42	2	4	380.0	Pending	2025-06-05 11:00:00
43	3	5	210.0	Delivered	2025-06-05 12:00:00
44	4	6	610.0	Delivered	2025-06-05 13:00:00
45	5	7	730.0	Cancelled	2025-06-05 14:00:00
46	6	8	190.0	Delivered	2025-06-05 15:00:00
47	7	9	310.0	Pending	2025-06-05 16:00:00
48	8	10	420.0	Delivered	2025-06-05 17:00:00
49	9	1	340.0	Delivered	2025-06-05 18:00:00
50	10	2	470.0	Pending	2025-06-05 19:00:00

Part 2 – Tasks

Task -1 Transactions (ACID) Using your accounts table:

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name='Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name='Bob';

ROLLBACK;
```

Query SQL

```
1 SELECT * FROM accounts;
```

Results

Query #1 Execution time: 0.16ms

id	name	balance
1	Alice	5000.0
2	Bob	3000.0

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name='Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name='Bob';

COMMIT;
```

Query SQL

```
1 SELECT * FROM accounts;
2
```

Results

Query #1

Execution time: 0.15ms

id	name	balance
1	Alice	4800.0
2	Bob	3200.0

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Ans:

The ACID property that guarantees money is never lost during a transfer is Atomicity. Atomicity ensures that all steps of the transaction are completed successfully or none of them are applied. If any error occurs, the entire transaction is rolled back, so no money is lost or partially transferred.

Task -2 Joins Using your customers and orders tables:

a) Write an INNER JOIN that lists every customer who has placed at least one order.

Query SQL

```
1 SELECT
2   customers.id,
3   customers.name,
4   customers.email,
5   orders.id AS order_id,
6   orders.total,
7   orders.status
8 FROM customers
9 INNER JOIN orders
10 ON customers.id = orders.customer_id;
11
```

Query #1 Execution time: 9.00ms

id	name	email	order_id	total	status
1	Dhruv	dhruv@gmail.com	1	350.0	Delivered
1	Dhruv	dhruv@gmail.com	21	390.0	Delivered
1	Dhruv	dhruv@gmail.com	41	250.0	Delivered
2	Aayush	aayush@gmail.com	2	450.0	Pending
2	Aayush	aayush@gmail.com	22	260.0	Pending
2	Aayush	aayush@gmail.com	42	380.0	Pending
3	Krish	krish@gmail.com	3	220.0	Delivered
3	Krish	krish@gmail.com	23	340.0	Delivered
3	Krish	krish@gmail.com	43	210.0	Delivered
4	Jaymin	jaymin@gmail.com	4	310.0	Cancelled
4	Jaymin	jaymin@gmail.com	24	190.0	Cancelled
4	Jaymin	jaymin@gmail.com	44	610.0	Delivered
5	Raj	raj@gmail.com	5	180.0	Delivered
5	Raj	raj@gmail.com	25	560.0	Delivered
5	Raj	raj@gmail.com	45	730.0	Cancelled
5	Raj	raj@gmail.com	45	730.0	Cancelled
6	Meet	meet@gmail.com	6	540.0	Pending
6	Meet	meet@gmail.com	26	690.0	Delivered
6	Meet	meet@gmail.com	46	190.0	Delivered
7	Harsh	harsh@gmail.com	7	600.0	Delivered
7	Harsh	harsh@gmail.com	27	170.0	Pending
7	Harsh	harsh@gmail.com	47	310.0	Pending
8	Yash	yash@gmail.com	8	150.0	Delivered
8	Yash	yash@gmail.com	28	290.0	Delivered
8	Yash	yash@gmail.com	48	420.0	Delivered
9	Dev	dev@gmail.com	9	270.0	Cancelled
9	Dev	dev@gmail.com	29	410.0	Cancelled
9	Dev	dev@gmail.com	49	340.0	Delivered
10	Parth	parth@gmail.com	10	320.0	Delivered
10	Parth	parth@gmail.com	30	350.0	Delivered

15	Karan	karan@gmail.com	15	240.0	Delivered
15	Karan	karan@gmail.com	35	590.0	Delivered
16	Om	om@gmail.com	16	630.0	Pending
16	Om	om@gmail.com	36	710.0	Pending
17	Vivek	vivek@gmail.com	17	720.0	Delivered
17	Vivek	vivek@gmail.com	37	180.0	Delivered
18	Rahul	rahul@gmail.com	18	160.0	Delivered
18	Rahul	rahul@gmail.com	38	300.0	Delivered
19	Sahil	sahil@gmail.com	19	330.0	Pending
19	Sahil	sahil@gmail.com	39	440.0	Pending
20	Ankit	ankit@gmail.com	20	450.0	Delivered
20	Ankit	ankit@gmail.com	40	360.0	Delivered

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

```
SELECT
  customers.id,
  customers.name,
  customers.email
FROM customers
LEFT JOIN orders
ON customers.id = orders.customer_id
WHERE orders.id IS NULL;
```

```
DELETE FROM orders
WHERE customer_id = 20;
```

Query #2 [Execution time: 0.00ms](#)

id	name	order_id	total
1	Dhruv	1	350.0
1	Dhruv	21	390.0
1	Dhruv	41	250.0
2	Aayush	2	450.0
2	Aayush	22	260.0
2	Aayush	42	380.0
3	Krish	3	220.0
3	Krish	23	340.0
3	Krish	43	210.0
4	Jaymin	4	310.0
4	Jaymin	24	190.0
4	Jaymin	44	610.0
5	Raj	5	180.0
5	Raj	25	560.0
5	Raj	45	730.0

6	Meet	6	540.0
6	Meet	26	690.0
6	Meet	46	190.0
7	Harsh	7	600.0
7	Harsh	27	170.0
7	Harsh	47	310.0
8	Yash	8	150.0
8	Yash	28	290.0
8	Yash	48	420.0
9	Dev	9	270.0
9	Dev	29	410.0
9	Dev	49	340.0
10	Parth	10	320.0
10	Parth	30	350.0

14	Aryan	14	410.0
14	Aryan	34	200.0
15	Karan	15	240.0
15	Karan	35	590.0
16	Om	16	630.0
16	Om	36	710.0
17	Vivek	17	720.0
17	Vivek	37	180.0
18	Rahul	18	160.0
18	Rahul	38	300.0
19	Sahil	19	330.0
19	Sahil	39	440.0
20	Ankit	20	450.0
20	Ankit	40	360.0

Query #3 Execution time: 0.26ms

id	name	email
----	------	-------

There are no results to be displayed.

Query #4 Execution time: 0.93ms

There are no results to be displayed.

Query #5 Execution time: 0.27ms

id	name	email
20	Ankit	ankit@gmail.com

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query.

-> The marketing team prefers the LEFT JOIN query because it includes customers who have never placed an order, making it easier to identify and target them with marketing campaigns.

Task - 3 Indexing Using your orders table:

- a) Pick a column that is not currently indexed (for example status or created_at).
- b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status = 'Delivered';
```

Query #1 Execution time: 3.3ms

EXPLAIN

-> Filter: (orders.'status' = 'Delivered') (cost=5.25 rows=5) (actual time=0.028..0.0466 rows=30 loops=1) -> Table scan on orders (cost=5.25 rows=50) (actual time=0.0203..0.031 rows=50 loops=1)

- c) Create an index on that column.

Query SQL

```
1 CREATE INDEX idx_status
2 ON orders(status);
```

Results

Query #1 Execution time: 21.94ms

There are no results to be displayed.

d) Re-run the exact same query and record the new numbers.

Query SQL

```

1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status = 'Delivered';

```

Query #1 Execution time: 0.39ms

EXPLAIN

-> Filter: (orders.'status' = 'Delivered') (cost=5.25 rows=5) (actual time=0.0286..0.0472 rows=30 loops=1) -> Table scan on orders (cost=5.25 rows=50) (actual time=0.0227..0.0334 rows=50 loops=1)

e) Submit a one-line comparison of before vs. after.

- Before indexing, MySQL performed a full table scan. After creating the index on the status column, MySQL used the index (idx_status), reducing the rows scanned and improving query performance.

TASK 4 Query Optimization

a) Rewrite it as a JOIN instead of a subquery.

Query SQL

```

1 SELECT
2     orders.*
3 FROM orders
4 INNER JOIN customers
5 ON orders.customer_id = customers.id
6 WHERE customers.signup_date >= '2025-01-10';

```

Query #1 Execution time: 0.54ms

id	customer_id	restaurant_id	total	status	created_at
10	10	10	320.0	Delivered	2025-06-01 19:00:00
30	10	1	350.0	Delivered	2025-06-03 19:00:00
50	10	2	470.0	Pending	2025-06-05 19:00:00
11	11	1	430.0	Pending	2025-06-02 10:00:00
31	11	2	480.0	Pending	2025-06-04 10:00:00
12	12	2	510.0	Delivered	2025-06-02 11:00:00
32	12	3	230.0	Delivered	2025-06-04 11:00:00
13	13	3	290.0	Delivered	2025-06-02 12:00:00
33	13	4	370.0	Delivered	2025-06-04 12:00:00
14	14	4	410.0	Cancelled	2025-06-02 13:00:00
34	14	5	200.0	Cancelled	2025-06-04 13:00:00
15	15	5	240.0	Delivered	2025-06-02 14:00:00
35	15	6	590.0	Delivered	2025-06-04 14:00:00
16	16	6	630.0	Pending	2025-06-02 15:00:00

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Query SQL

```
1 SELECT
2   orders.id,
3   customers.name,
4   orders.total,
5   orders.status,
6   orders.created_at
7 FROM orders
8 INNER JOIN customers
9 ON orders.customer_id = customers.id
10 WHERE customers.signup_date >= '2025-01-10';
```

Query #1 Execution time: 5.47ms

id	name	total	status	created_at
10	Parth	320.0	Delivered	2025-06-01 19:00:00
30	Parth	350.0	Delivered	2025-06-03 19:00:00
50	Parth	470.0	Pending	2025-06-05 19:00:00
11	Nisarg	430.0	Pending	2025-06-02 10:00:00
31	Nisarg	480.0	Pending	2025-06-04 10:00:00
12	Manav	510.0	Delivered	2025-06-02 11:00:00
32	Manav	230.0	Delivered	2025-06-04 11:00:00
13	Rohan	290.0	Delivered	2025-06-02 12:00:00
33	Rohan	370.0	Delivered	2025-06-04 12:00:00
14	Aryan	410.0	Cancelled	2025-06-02 13:00:00
34	Aryan	200.0	Cancelled	2025-06-04 13:00:00
15	Karan	240.0	Delivered	2025-06-02 14:00:00
35	Karan	590.0	Delivered	2025-06-04 14:00:00
16	Om	630.0	Pending	2025-06-02 15:00:00

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE customer_id IN (
5   SELECT id
6   FROM customers
7   WHERE signup_date >= '2025-01-10'
8 );
```

Query #1 Execution time: 14.36ms

EXPLAIN

-> Nested loop inner join (cost=4.37 rows=6.33) (actual time=0.0545..0.103 rows=23 loops=1) -> Filter: (customers.signup_date >= DATE'2025-01-10') (cost=2.15 rows=6.33) (actual time=0.026..0.0309 rows=11 loops=1) -> Table scan on customers (cost=2.15 rows=19) (actual time=0.0195..0.027 rows=20 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.266 rows=1) (actual time=0.00507..0.00572 rows=2.09 loops=11)

TASK 5 The Customer Profile Page

Query SQL

```

1 SELECT
2     restaurants.name AS restaurant_name,
3     orders.total,
4     orders.status,
5     orders.created_at
6 FROM orders
7 INNER JOIN restaurants
8 ON orders.restaurant_id = restaurants.id
9 WHERE orders.customer_id = 1
10 ORDER BY orders.created_at DESC
11 LIMIT 5;

```

Results

Copy as

Query #1

Execution time: 0.36ms

restaurant_name	total	status	created_at
McDonalds	250.0	Delivered	2025-06-05 10:00:00
Pizza Hut	390.0	Delivered	2025-06-03 10:00:00
Dominos	350.0	Delivered	2025-06-01 10:00:00

Query SQL

```

1 EXPLAIN ANALYZE
2 SELECT
3     restaurants.name AS restaurant_name,
4     orders.total,
5     orders.status,
6     orders.created_at
7 FROM orders
8 INNER JOIN restaurants
9 ON orders.restaurant_id = restaurants.id
10 WHERE orders.customer_id = 1
11 ORDER BY orders.created_at DESC
12 LIMIT 5;

```

Query #1

Execution time: 0.51ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.037..0.0413 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0336..0.0376 rows=3 loops=1) -> Sort: orders.created_at DESC (cost=1.05 rows=3) (actual time=0.0281..0.0286 rows=3 loops=1) -> Filter: (orders.restaurant_id is not null) (cost=1.05 rows=3) (actual time=0.0159..0.018 rows=3 loops=1) -> Index lookup on orders using customer_id (customer_id = 1) (cost=1.05 rows=3) (actual time=0.0148..0.0166 rows=3 loops=1) -> Single-row index lookup on restaurants using PRIMARY (id = orders.restaurant_id) (cost=0.283 rows=1) (actual time=0.00237..0.00242 rows=1 loops=3)

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

```

13 CREATE INDEX idx_customer_created
14 ON orders(customer_id, created_at);

```

Query #2 Execution time: 17.85ms

There are no results to be displayed.

d) Re-run the same query and compare the new numbers to step (b).

Query SQL

```

1 EXPLAIN
2 SELECT
3     restaurants.name AS restaurant_name,
4     orders.total,
5     orders.status,
6     orders.created_at
7 FROM orders
8 INNER JOIN restaurants
9 ON orders.restaurant_id = restaurants.id
10 WHERE orders.customer_id = 1
11 ORDER BY orders.created_at DESC
12 LIMIT 5;

```

Query #1 Execution time: 0.47ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.1 rows=3) -> Nested loop inner join (cost=2.1 rows=3) -> Sort: orders.created_at DESC (cost=1.05 rows=3) -> Filter: (orders.restaurant_id is not null) (cost=1.05 rows=3) -> Index lookup on orders using customer_id (customer_id = 1) (cost=1.05 rows=3) -> Single-row index lookup on restaurants using PRIMARY (id = orders.restaurant_id) (cost=0.283 rows=1)

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Query SQL ●

```

1 SELECT
2     restaurants.name AS restaurant_name,
3     orders.total,
4     orders.status,
5     orders.created_at
6 FROM orders
7 INNER JOIN restaurants
8 ON orders.restaurant_id = restaurants.id
9 WHERE orders.customer_id = 1
10 ORDER BY orders.created_at DESC
11 LIMIT 5;

```

Results

Query #1 Execution time: 0.39ms

restaurant_name	total	status	created_at
McDonalds	250.0	Delivered	2025-06-05 10:00:00
Pizza Hut	390.0	Delivered	2025-06-03 10:00:00
Dominos	350.0	Delivered	2025-06-01 10:00:00

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Before Index execution time

0.34 ms

After Index execution time

0.25 ms

-I created a composite index on (customer_id, created_at) because the query filters by customer_id and sorts by created_at. This index allows MySQL to quickly locate the required customer's orders and return them in the correct order without scanning the entire table.

- As a result, the execution time improved from 0.34 ms to 0.25 ms, making the customer profile page load faster and improving database performance as the number of orders grows.